

HerAI Fellowship

INFRASTRUCTURE FOR AI

Modul Belajar Praktis dan Mudah Dipahami

Merancang fondasi komputasi yang aman, terukur, andal, dan efisien untuk workload AI.

|                  |                                                        |
|------------------|--------------------------------------------------------|
| Level            | Intermediate                                           |
| Estimasi belajar | 18-22 jam                                              |
| Prasyarat        | Linux dasar, jaringan dasar, dan pemahaman aplikasi AI |
| Versi            | 1.0 - 23 Juli 2026                                     |
| Route            | /participant/courses/infrastructure-for-ai             |

Disusun untuk Participant Module - Project HerAI

# Tentang Modul

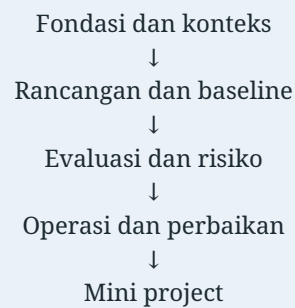
Infrastructure for AI menjelaskan bagaimana compute, accelerator, storage, network, container, cloud, orchestration, security, dan observability disatukan. Peserta belajar memilih arsitektur berdasarkan workload, risiko, dan biaya, bukan berdasarkan tren.

**Pola belajar:** pahami tujuan, buat baseline kecil, uji kegagalan, dokumentasikan bukti, lalu perbaiki secara bertahap. Jangan menambah kompleksitas sebelum masalah dan metriknya jelas.

## Cara Menggunakan Modul

- Baca tujuan dan gambaran sederhana setiap bab.
- Kerjakan checkpoint tanpa melihat kembali materi.
- Adaptasi contoh pada data aman atau sintetis.
- Catat asumsi, error, keputusan, dan keterbatasan.
- Gunakan mini project sebagai artefak portofolio.

## Peta Belajar



# Daftar Isi

---

- 01 Bab 1 - Arsitektur Infrastruktur AI**
- 02 Bab 2 - CPU, GPU, Accelerator, dan Memori
- 03 Bab 3 - Storage dan Siklus Hidup Data
- 04 Bab 4 - Jaringan untuk Workload AI
- 05 Bab 5 - Virtualization dan Container
- 06 Bab 6 - Cloud, On-Premise, dan Hybrid
- 07 Bab 7 - Orkestrasi dengan Kubernetes
- 08 Bab 8 - Infrastructure as Code
- 09 Bab 9 - IAM, Secret, dan Supply Chain Security
- 10 Bab 10 - Observability, Reliability, dan FinOps
- 11 Bab 11 - Mini Project: Platform Layanan AI Internal**
- 12 Bab 12 - Kuis Akhir**
- 13 Bab 13 - Diskusi dan Refleksi
- 14 Bab 14 - Glosarium dan Checklist
- 15 Bab 15 - Ringkasan dan Referensi**

**HerAI Fellowship - Participant Module** Merancang fondasi komputasi yang aman, terukur, andal, dan efisien untuk workload AI.

## Tentang Modul

Infrastructure for AI menjelaskan bagaimana compute, accelerator, storage, network, container, cloud, orchestration, security, dan observability disatukan. Peserta belajar memilih arsitektur berdasarkan workload, risiko, dan biaya, bukan berdasarkan tren.

Modul disusun dengan bahasa bertahap. Setiap bab berisi tujuan, konsep inti, alur kerja, contoh kasus, kesalahan umum, checkpoint, dan latihan. Peserta tidak perlu menghafal seluruh istilah. Yang lebih penting adalah memahami hubungan antarkomponen dan mampu menjelaskan alasan di balik sebuah pilihan.

## Capaian Pembelajaran

Setelah menyelesaikan modul, peserta mampu:

- menjelaskan fondasi dan istilah utama Infrastructure for AI;
- menerjemahkan kebutuhan menjadi rancangan yang dapat diuji;
- membuat prototipe kecil dengan dokumentasi dan kontrol dasar;
- mengevaluasi kualitas, risiko, keterbatasan, serta biaya;
- menyampaikan hasil dan rekomendasi kepada pemangku kepentingan.

## Prasyarat

- Linux dasar, jaringan dasar, dan pemahaman aplikasi AI.
- Mampu membaca diagram sederhana dan mengikuti contoh kode.
- Bersedia mencatat asumsi, kegagalan, dan keputusan selama latihan.

## Cara Belajar yang Disarankan

1. Baca gambaran dan tujuan setiap bab.
2. Buat catatan istilah dengan bahasa sendiri.
3. Kerjakan checkpoint sebelum melihat kembali materi.
4. Jalankan atau adaptasi contoh kode bila relevan.
5. Gunakan mini project sebagai portofolio, bukan sekadar tugas akhir.
6. Lakukan review keamanan, privasi, dan dampak sebelum demonstrasi.

## Peta Materi

| Tahap       | Fokus                                    | Hasil                         |
|-------------|------------------------------------------|-------------------------------|
| Fondasi     | Istilah, tujuan, konteks, dan arsitektur | Peta masalah yang jelas       |
| Pembangunan | Data, proses, komponen, dan integrasi    | Baseline yang dapat diuji     |
| Evaluasi    | Kualitas, risiko, subgroup, biaya        | Bukti dan daftar keterbatasan |
| Operasi     | Monitoring, ownership, respons, iterasi  | Sistem yang dapat dipelihara  |

# Bab 1 - Arsitektur Infrastruktur AI

## Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran arsitektur infrastruktur ai dalam Infrastructure for AI;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

## Gambaran Sederhana

Arsitektur Infrastruktur AI tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

**Analogi:** Infrastruktur AI seperti fasilitas sebuah kota: model adalah aktivitas di dalamnya, sedangkan listrik, jalan, gudang, keamanan, dan pusat kendali menentukan apakah aktivitas dapat berlangsung stabil.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

## Konsep Inti

| Istilah                           | Penjelasan mudah                                                                                        | Cara memeriksa pemahaman                                                                              |
|-----------------------------------|---------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <b>workload</b>                   | konsep penting dalam arsitektur infrastruktur ai yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana workload diukur, diuji, atau dikendalikan?                   |
| <b>training</b>                   | konsep penting dalam arsitektur infrastruktur ai yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana training diukur, diuji, atau dikendalikan?                   |
| <b>inference</b>                  | konsep penting dalam arsitektur infrastruktur ai yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana inference diukur, diuji, atau dikendalikan?                  |
| <b>control plane</b>              | konsep penting dalam arsitektur infrastruktur ai yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana control plane diukur, diuji, atau dikendalikan?              |
| <b>data plane</b>                 | konsep penting dalam arsitektur infrastruktur ai yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana data plane diukur, diuji, atau dikendalikan?                 |
| <b>non-functional requirement</b> | konsep penting dalam arsitektur infrastruktur ai yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana non-functional requirement diukur, diuji, atau dikendalikan? |

## Hubungan antarkonsep

Bayangkan alur **workload** -> **training** -> **inference** -> **control plane**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **workload** dapat memengaruhi **training**, lalu mengubah cara **inference** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

## Langkah Kerja

1. Tentukan tujuan arsitektur infrastruktur ai dan keputusan yang akan dipengaruhi.

2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk workload serta training.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

## Contoh Kasus

Dalam mini project **Platform Layanan AI Internal**, tim perlu menerapkan arsitektur infrastruktur ai secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **workload** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

| Situasi                       | Pilihan tindakan                              | Trade-off                                               |
|-------------------------------|-----------------------------------------------|---------------------------------------------------------|
| Data atau input belum lengkap | Tolak, minta perbaikan, atau gunakan fallback | Menjaga kualitas tetapi dapat menambah friksi           |
| Hasil belum pasti             | Tampilkan ketidakpastian dan minta review     | Lebih aman tetapi membutuhkan waktu manusia             |
| Beban meningkat               | Scale, antrekan, atau pembatasan              | Menjaga layanan tetapi menambah biaya atau waktu tunggu |
| Perubahan berisiko            | Uji terbatas dan siapkan rollback             | Rilis lebih lambat tetapi dampak kegagalan lebih kecil  |

## Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan arsitektur infrastruktur ai.
- Menganggap workload sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

## Checkpoint

1. Jelaskan workload dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara training dan inference dalam bab ini?
3. Sebutkan satu risiko ketika arsitektur infrastruktur ai diterapkan tanpa pengujian.

## Latihan

1. Buat diagram sederhana yang menghubungkan workload, training, inference, control plane. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

## Bab 2 - CPU, GPU, Accelerator, dan Memori

### Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran cpu, gpu, accelerator, dan memori dalam Infrastructure for AI;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

### Gambaran Sederhana

CPU, GPU, Accelerator, dan Memori tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

**Analogi:** Infrastruktur AI seperti fasilitas sebuah kota: model adalah aktivitas di dalamnya, sedangkan listrik, jalan, gudang, keamanan, dan pusat kendali menentukan apakah aktivitas dapat berlangsung stabil.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

### Konsep Inti

| Istilah            | Penjelasan mudah                                                                                              | Cara memeriksa pemahaman                                                               |
|--------------------|---------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <b>CPU</b>         | konsep penting dalam cpu, gpu, accelerator, dan memori yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana CPU diukur, diuji, atau dikendalikan?         |
| <b>GPU</b>         | konsep penting dalam cpu, gpu, accelerator, dan memori yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana GPU diukur, diuji, atau dikendalikan?         |
| <b>accelerator</b> | konsep penting dalam cpu, gpu, accelerator, dan memori yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana accelerator diukur, diuji, atau dikendalikan? |
| <b>VRAM</b>        | konsep penting dalam cpu, gpu, accelerator, dan memori yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana VRAM diukur, diuji, atau dikendalikan?        |
| <b>bandwidth</b>   | konsep penting dalam cpu, gpu, accelerator, dan memori yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana bandwidth diukur, diuji, atau dikendalikan?   |
| <b>precision</b>   | konsep penting dalam cpu, gpu, accelerator, dan memori yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana precision diukur, diuji, atau dikendalikan?   |

### Hubungan antarkonsep

Bayangkan alur **CPU -> GPU -> accelerator -> VRAM**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **CPU** dapat memengaruhi **GPU**, lalu mengubah cara **accelerator** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

## Langkah Kerja

1. Tentukan tujuan cpu, gpu, accelerator, dan memori dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk CPU serta GPU.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

## Contoh Kasus

Dalam mini project **Platform Layanan AI Internal**, tim perlu menerapkan cpu, gpu, accelerator, dan memori secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **CPU** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

| Situasi                       | Pilihan tindakan                              | Trade-off                                               |
|-------------------------------|-----------------------------------------------|---------------------------------------------------------|
| Data atau input belum lengkap | Tolak, minta perbaikan, atau gunakan fallback | Menjaga kualitas tetapi dapat menambah friksi           |
| Hasil belum pasti             | Tampilkan ketidakpastian dan minta review     | Lebih aman tetapi membutuhkan waktu manusia             |
| Beban meningkat               | Scale, antrekan, atau pembatasan              | Menjaga layanan tetapi menambah biaya atau waktu tunggu |
| Perubahan berisiko            | Uji terbatas dan siapkan rollback             | Rilis lebih lambat tetapi dampak kegagalan lebih kecil  |

## Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan cpu, gpu, accelerator, dan memori.
- Menganggap CPU sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

## Checkpoint

1. Jelaskan CPU dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara GPU dan accelerator dalam bab ini?
3. Sebutkan satu risiko ketika cpu, gpu, accelerator, dan memori diterapkan tanpa pengujian.

## Latihan

1. Buat diagram sederhana yang menghubungkan CPU, GPU, accelerator, VRAM. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.



## Bab 3 - Storage dan Siklus Hidup Data

### Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran storage dan siklus hidup data dalam Infrastructure for AI;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

### Gambaran Sederhana

Storage dan Siklus Hidup Data tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan:

**keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

**Analogi:** Infrastruktur AI seperti fasilitas sebuah kota: model adalah aktivitas di dalamnya, sedangkan listrik, jalan, gudang, keamanan, dan pusat kendali menentukan apakah aktivitas dapat berlangsung stabil.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

### Konsep Inti

| Istilah               | Penjelasan mudah                                                                                          | Cara memeriksa pemahaman                                                                  |
|-----------------------|-----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>object storage</b> | konsep penting dalam storage dan siklus hidup data yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana object storage diukur, diuji, atau dikendalikan? |
| <b>block</b>          | konsep penting dalam storage dan siklus hidup data yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana block diukur, diuji, atau dikendalikan?          |
| <b>file</b>           | konsep penting dalam storage dan siklus hidup data yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana file diukur, diuji, atau dikendalikan?           |
| <b>IOPS</b>           | konsep penting dalam storage dan siklus hidup data yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana IOPS diukur, diuji, atau dikendalikan?           |
| <b>throughput</b>     | jumlah pekerjaan yang dapat diselesaikan per satuan waktu                                                 | Pertanyaan yang perlu dijawab: bagaimana throughput diukur, diuji, atau dikendalikan?     |
| <b>tiering</b>        | konsep penting dalam storage dan siklus hidup data yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana tiering diukur, diuji, atau dikendalikan?        |

### Hubungan antarkonsep

Bayangkan alur **object storage** -> **block** -> **file** -> **IOPS**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **object storage** dapat memengaruhi **block**, lalu mengubah cara **file** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

### Langkah Kerja

1. Tentukan tujuan storage dan siklus hidup data dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk object storage serta block.

4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

### Contoh Kasus

Dalam mini project **Platform Layanan AI Internal**, tim perlu menerapkan storage dan siklus hidup data secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **object storage** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

| Situasi                       | Pilihan tindakan                              | Trade-off                                               |
|-------------------------------|-----------------------------------------------|---------------------------------------------------------|
| Data atau input belum lengkap | Tolak, minta perbaikan, atau gunakan fallback | Menjaga kualitas tetapi dapat menambah friksi           |
| Hasil belum pasti             | Tampilkan ketidakpastian dan minta review     | Lebih aman tetapi membutuhkan waktu manusia             |
| Beban meningkat               | Scale, antrekan, atau pembatasan              | Menjaga layanan tetapi menambah biaya atau waktu tunggu |
| Perubahan berisiko            | Uji terbatas dan siapkan rollback             | Rilis lebih lambat tetapi dampak kegagalan lebih kecil  |

### Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan storage dan siklus hidup data.
- Menganggap object storage sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

### Checkpoint

1. Jelaskan object storage dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara block dan file dalam bab ini?
3. Sebutkan satu risiko ketika storage dan siklus hidup data diterapkan tanpa pengujian.

### Latihan

1. Buat diagram sederhana yang menghubungkan object storage, block, file, IOPS. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

## Bab 4 - Jaringan untuk Workload AI

### Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran jaringan untuk workload ai dalam Infrastructure for AI;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

### Gambaran Sederhana

Jaringan untuk Workload AI tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan:

**keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

**Analogi:** Infrastruktur AI seperti fasilitas sebuah kota: model adalah aktivitas di dalamnya, sedangkan listrik, jalan, gudang, keamanan, dan pusat kendali menentukan apakah aktivitas dapat berlangsung stabil.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

### Konsep Inti

| Istilah                | Penjelasan mudah                                                                                       | Cara memeriksa pemahaman                                                                   |
|------------------------|--------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>latency</b>         | waktu yang dibutuhkan sistem untuk menghasilkan respons                                                | Pertanyaan yang perlu dijawab: bagaimana latency diukur, diuji, atau dikendalikan?         |
| <b>bandwidth</b>       | konsep penting dalam jaringan untuk workload ai yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana bandwidth diukur, diuji, atau dikendalikan?       |
| <b>topology</b>        | konsep penting dalam jaringan untuk workload ai yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana topology diukur, diuji, atau dikendalikan?        |
| <b>load balancer</b>   | konsep penting dalam jaringan untuk workload ai yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana load balancer diukur, diuji, atau dikendalikan?   |
| <b>egress</b>          | konsep penting dalam jaringan untuk workload ai yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana egress diukur, diuji, atau dikendalikan?          |
| <b>private network</b> | konsep penting dalam jaringan untuk workload ai yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana private network diukur, diuji, atau dikendalikan? |

### Hubungan antarkonsep

Bayangkan alur **latency** -> **bandwidth** -> **topology** -> **load balancer**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **latency** dapat memengaruhi **bandwidth**, lalu mengubah cara **topology** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

### Langkah Kerja

1. Tentukan tujuan jaringan untuk workload ai dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.

3. Tetapkan definisi dan kriteria penerimaan untuk latency serta bandwidth.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

## Contoh Kasus

Dalam mini project **Platform Layanan AI Internal**, tim perlu menerapkan jaringan untuk workload ai secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **latency** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

| Situasi                       | Pilihan tindakan                              | Trade-off                                               |
|-------------------------------|-----------------------------------------------|---------------------------------------------------------|
| Data atau input belum lengkap | Tolak, minta perbaikan, atau gunakan fallback | Menjaga kualitas tetapi dapat menambah friksi           |
| Hasil belum pasti             | Tampilkan ketidakpastian dan minta review     | Lebih aman tetapi membutuhkan waktu manusia             |
| Beban meningkat               | Scale, antrekan, atau pembatasan              | Menjaga layanan tetapi menambah biaya atau waktu tunggu |
| Perubahan berisiko            | Uji terbatas dan siapkan rollback             | Rilis lebih lambat tetapi dampak kegagalan lebih kecil  |

## Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan jaringan untuk workload ai.
- Menganggap latency sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

## Checkpoint

1. Jelaskan latency dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara bandwidth dan topology dalam bab ini?
3. Sebutkan satu risiko ketika jaringan untuk workload ai diterapkan tanpa pengujian.

## Latihan

1. Buat diagram sederhana yang menghubungkan latency, bandwidth, topology, load balancer. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

## Bab 5 - Virtualization dan Container

### Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran virtualization dan container dalam Infrastructure for AI;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

### Gambaran Sederhana

Virtualization dan Container tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan:

**keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

**Analogi:** Infrastruktur AI seperti fasilitas sebuah kota: model adalah aktivitas di dalamnya, sedangkan listrik, jalan, gudang, keamanan, dan pusat kendali menentukan apakah aktivitas dapat berlangsung stabil.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

### Konsep Inti

| Istilah                | Penjelasan mudah                                                                                         | Cara memeriksa pemahaman                                                                   |
|------------------------|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>virtual machine</b> | konsep penting dalam virtualization dan container yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana virtual machine diukur, diuji, atau dikendalikan? |
| <b>container</b>       | konsep penting dalam virtualization dan container yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana container diukur, diuji, atau dikendalikan?       |
| <b>image</b>           | konsep penting dalam virtualization dan container yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana image diukur, diuji, atau dikendalikan?           |
| <b>runtime</b>         | konsep penting dalam virtualization dan container yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana runtime diukur, diuji, atau dikendalikan?         |
| <b>isolation</b>       | konsep penting dalam virtualization dan container yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana isolation diukur, diuji, atau dikendalikan?       |
| <b>registry</b>        | konsep penting dalam virtualization dan container yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana registry diukur, diuji, atau dikendalikan?        |

### Hubungan antarkonsep

Bayangkan alur **virtual machine** -> **container** -> **image** -> **runtime**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **virtual machine** dapat memengaruhi **container**, lalu mengubah cara **image** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

### Langkah Kerja

1. Tentukan tujuan virtualization dan container dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.

3. Tetapkan definisi dan kriteria penerimaan untuk virtual machine serta container.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

### Contoh Kasus

Dalam mini project **Platform Layanan AI Internal**, tim perlu menerapkan virtualization dan container secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **virtual machine** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

| Situasi                       | Pilihan tindakan                              | Trade-off                                               |
|-------------------------------|-----------------------------------------------|---------------------------------------------------------|
| Data atau input belum lengkap | Tolak, minta perbaikan, atau gunakan fallback | Menjaga kualitas tetapi dapat menambah friksi           |
| Hasil belum pasti             | Tampilkan ketidakpastian dan minta review     | Lebih aman tetapi membutuhkan waktu manusia             |
| Beban meningkat               | Scale, antrekan, atau pembatasan              | Menjaga layanan tetapi menambah biaya atau waktu tunggu |
| Perubahan berisiko            | Uji terbatas dan siapkan rollback             | Rilis lebih lambat tetapi dampak kegagalan lebih kecil  |

### Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan virtualization dan container.
- Menganggap virtual machine sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

### Checkpoint

1. Jelaskan virtual machine dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara container dan image dalam bab ini?
3. Sebutkan satu risiko ketika virtualization dan container diterapkan tanpa pengujian.

### Latihan

1. Buat diagram sederhana yang menghubungkan virtual machine, container, image, runtime. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

## Bab 6 - Cloud, On-Premise, dan Hybrid

### Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran cloud, on-premise, dan hybrid dalam Infrastructure for AI;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

### Gambaran Sederhana

Cloud, On-Premise, dan Hybrid tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan:

**keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

**Analogi:** Infrastruktur AI seperti fasilitas sebuah kota: model adalah aktivitas di dalamnya, sedangkan listrik, jalan, gudang, keamanan, dan pusat kendali menentukan apakah aktivitas dapat berlangsung stabil.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

### Konsep Inti

| Istilah                      | Penjelasan mudah                                                                                          | Cara memeriksa pemahaman                                                                         |
|------------------------------|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| <b>cloud</b>                 | konsep penting dalam cloud, on-premise, dan hybrid yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana cloud diukur, diuji, atau dikendalikan?                 |
| <b>on-premise</b>            | konsep penting dalam cloud, on-premise, dan hybrid yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana on-premise diukur, diuji, atau dikendalikan?            |
| <b>hybrid</b>                | konsep penting dalam cloud, on-premise, dan hybrid yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana hybrid diukur, diuji, atau dikendalikan?                |
| <b>elasticity</b>            | konsep penting dalam cloud, on-premise, dan hybrid yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana elasticity diukur, diuji, atau dikendalikan?            |
| <b>sovereignty</b>           | konsep penting dalam cloud, on-premise, dan hybrid yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana sovereignty diukur, diuji, atau dikendalikan?           |
| <b>shared responsibility</b> | konsep penting dalam cloud, on-premise, dan hybrid yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana shared responsibility diukur, diuji, atau dikendalikan? |

### Hubungan antarkonsep

Bayangkan alur **cloud** -> **on-premise** -> **hybrid** -> **elasticity**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku. Contohnya, perubahan pada **cloud** dapat memengaruhi **on-premise**, lalu mengubah cara **hybrid** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

### Langkah Kerja

1. Tentukan tujuan cloud, on-premise, dan hybrid dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk cloud serta on-premise.



4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

### Contoh Kasus

Dalam mini project **Platform Layanan AI Internal**, tim perlu menerapkan cloud, on-premise, dan hybrid secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **cloud** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

| Situasi                       | Pilihan tindakan                              | Trade-off                                               |
|-------------------------------|-----------------------------------------------|---------------------------------------------------------|
| Data atau input belum lengkap | Tolak, minta perbaikan, atau gunakan fallback | Menjaga kualitas tetapi dapat menambah friksi           |
| Hasil belum pasti             | Tampilkan ketidakpastian dan minta review     | Lebih aman tetapi membutuhkan waktu manusia             |
| Beban meningkat               | Scale, antrekan, atau pembatasan              | Menjaga layanan tetapi menambah biaya atau waktu tunggu |
| Perubahan berisiko            | Uji terbatas dan siapkan rollback             | Rilis lebih lambat tetapi dampak kegagalan lebih kecil  |

### Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan cloud, on-premise, dan hybrid.
- Menganggap cloud sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

### Checkpoint

1. Jelaskan cloud dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara on-premise dan hybrid dalam bab ini?
3. Sebutkan satu risiko ketika cloud, on-premise, dan hybrid diterapkan tanpa pengujian.

### Latihan

1. Buat diagram sederhana yang menghubungkan cloud, on-premise, hybrid, elasticity. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.



## Bab 7 - Orkestrasi dengan Kubernetes

### Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran orkestrasi dengan kubernetes dalam Infrastructure for AI;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

### Gambaran Sederhana

Orkestrasi dengan Kubernetes tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan:

**keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

**Analogi:** Infrastruktur AI seperti fasilitas sebuah kota: model adalah aktivitas di dalamnya, sedangkan listrik, jalan, gudang, keamanan, dan pusat kendali menentukan apakah aktivitas dapat berlangsung stabil.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

### Konsep Inti

| Istilah               | Penjelasan mudah                                                                                         | Cara memeriksa pemahaman                                                                  |
|-----------------------|----------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>pod</b>            | konsep penting dalam orkestrasi dengan kubernetes yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana pod diukur, diuji, atau dikendalikan?            |
| <b>deployment</b>     | konsep penting dalam orkestrasi dengan kubernetes yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana deployment diukur, diuji, atau dikendalikan?     |
| <b>service</b>        | konsep penting dalam orkestrasi dengan kubernetes yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana service diukur, diuji, atau dikendalikan?        |
| <b>job</b>            | konsep penting dalam orkestrasi dengan kubernetes yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana job diukur, diuji, atau dikendalikan?            |
| <b>autoscaling</b>    | konsep penting dalam orkestrasi dengan kubernetes yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana autoscaling diukur, diuji, atau dikendalikan?    |
| <b>resource quota</b> | konsep penting dalam orkestrasi dengan kubernetes yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana resource quota diukur, diuji, atau dikendalikan? |

### Hubungan antarkonsep

Bayangkan alur **pod** -> **deployment** -> **service** -> **job**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **pod** dapat memengaruhi **deployment**, lalu mengubah cara **service** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

### Langkah Kerja

1. Tentukan tujuan orkestrasi dengan kubernetes dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk pod serta deployment.

4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

### Contoh Kasus

Dalam mini project **Platform Layanan AI Internal**, tim perlu menerapkan orkestrasi dengan kubernetes secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **pod** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

| Situasi                       | Pilihan tindakan                              | Trade-off                                               |
|-------------------------------|-----------------------------------------------|---------------------------------------------------------|
| Data atau input belum lengkap | Tolak, minta perbaikan, atau gunakan fallback | Menjaga kualitas tetapi dapat menambah friksi           |
| Hasil belum pasti             | Tampilkan ketidakpastian dan minta review     | Lebih aman tetapi membutuhkan waktu manusia             |
| Beban meningkat               | Scale, antrekan, atau pembatasan              | Menjaga layanan tetapi menambah biaya atau waktu tunggu |
| Perubahan berisiko            | Uji terbatas dan siapkan rollback             | Rilis lebih lambat tetapi dampak kegagalan lebih kecil  |

### Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan orkestrasi dengan kubernetes.
- Menganggap pod sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

### Checkpoint

1. Jelaskan pod dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara deployment dan service dalam bab ini?
3. Sebutkan satu risiko ketika orkestrasi dengan kubernetes diterapkan tanpa pengujian.

### Latihan

1. Buat diagram sederhana yang menghubungkan pod, deployment, service, job. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

## Bab 8 - Infrastructure as Code

### Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran infrastructure as code dalam Infrastructure for AI;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

### Gambaran Sederhana

Infrastructure as Code tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

**Analogi:** Infrastruktur AI seperti fasilitas sebuah kota: model adalah aktivitas di dalamnya, sedangkan listrik, jalan, gudang, keamanan, dan pusat kendali menentukan apakah aktivitas dapat berlangsung stabil.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

### Konsep Inti

| Istilah            | Penjelasan mudah                                                                                   | Cara memeriksa pemahaman                                                               |
|--------------------|----------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <b>declarative</b> | konsep penting dalam infrastructure as code yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana declarative diukur, diuji, atau dikendalikan? |
| <b>Terraform</b>   | konsep penting dalam infrastructure as code yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana Terraform diukur, diuji, atau dikendalikan?   |
| <b>module</b>      | konsep penting dalam infrastructure as code yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana module diukur, diuji, atau dikendalikan?      |
| <b>state</b>       | konsep penting dalam infrastructure as code yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana state diukur, diuji, atau dikendalikan?       |
| <b>review</b>      | konsep penting dalam infrastructure as code yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana review diukur, diuji, atau dikendalikan?      |
| <b>drift</b>       | perubahan distribusi data atau hubungan yang dipelajari model                                      | Pertanyaan yang perlu dijawab: bagaimana drift diukur, diuji, atau dikendalikan?       |

### Hubungan antarkonsep

Bayangkan alur **declarative** -> **Terraform** -> **module** -> **state**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku. Contohnya, perubahan pada **declarative** dapat memengaruhi **Terraform**, lalu mengubah cara **module** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

### Langkah Kerja

1. Tentukan tujuan infrastructure as code dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk declarative serta Terraform.
4. Bangun versi kecil menggunakan data atau skenario yang aman.

5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

## Contoh Kasus

Dalam mini project **Platform Layanan AI Internal**, tim perlu menerapkan infrastructure as code secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **declarative** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

| Situasi                       | Pilihan tindakan                              | Trade-off                                               |
|-------------------------------|-----------------------------------------------|---------------------------------------------------------|
| Data atau input belum lengkap | Tolak, minta perbaikan, atau gunakan fallback | Menjaga kualitas tetapi dapat menambah friksi           |
| Hasil belum pasti             | Tampilkan ketidakpastian dan minta review     | Lebih aman tetapi membutuhkan waktu manusia             |
| Beban meningkat               | Scale, antrekan, atau pembatasan              | Menjaga layanan tetapi menambah biaya atau waktu tunggu |
| Perubahan berisiko            | Uji terbatas dan siapkan rollback             | Rilis lebih lambat tetapi dampak kegagalan lebih kecil  |

## Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan infrastructure as code.
- Menganggap declarative sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

## Checkpoint

1. Jelaskan declarative dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara Terraform dan module dalam bab ini?
3. Sebutkan satu risiko ketika infrastructure as code diterapkan tanpa pengujian.

## Latihan

1. Buat diagram sederhana yang menghubungkan declarative, Terraform, module, state. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

## Bab 9 - IAM, Secret, dan Supply Chain Security

### Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran iam, secret, dan supply chain security dalam Infrastructure for AI;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

### Gambaran Sederhana

IAM, Secret, dan Supply Chain Security tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

**Analogi:** Infrastruktur AI seperti fasilitas sebuah kota: model adalah aktivitas di dalamnya, sedangkan listrik, jalan, gudang, keamanan, dan pusat kendali menentukan apakah aktivitas dapat berlangsung stabil.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

### Konsep Inti

| Istilah                | Penjelasan mudah                                                                                                   | Cara memeriksa pemahaman                                                                   |
|------------------------|--------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>identity</b>        | konsep penting dalam iam, secret, dan supply chain security yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana identity diukur, diuji, atau dikendalikan?        |
| <b>role</b>            | konsep penting dalam iam, secret, dan supply chain security yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana role diukur, diuji, atau dikendalikan?            |
| <b>secret</b>          | konsep penting dalam iam, secret, dan supply chain security yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana secret diukur, diuji, atau dikendalikan?          |
| <b>signing</b>         | konsep penting dalam iam, secret, dan supply chain security yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana signing diukur, diuji, atau dikendalikan?         |
| <b>vulnerability</b>   | konsep penting dalam iam, secret, dan supply chain security yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana vulnerability diukur, diuji, atau dikendalikan?   |
| <b>least privilege</b> | pemberian akses minimum yang benar-benar diperlukan                                                                | Pertanyaan yang perlu dijawab: bagaimana least privilege diukur, diuji, atau dikendalikan? |

### Hubungan antarkonsep

Bayangkan alur **identity** -> **role** -> **secret** -> **signing**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **identity** dapat memengaruhi **role**, lalu mengubah cara **secret** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.

### Langkah Kerja

1. Tentukan tujuan iam, secret, dan supply chain security dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk identity serta role.

4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

### Contoh Kasus

Dalam mini project **Platform Layanan AI Internal**, tim perlu menerapkan iam, secret, dan supply chain security secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **identity** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

| Situasi                       | Pilihan tindakan                              | Trade-off                                               |
|-------------------------------|-----------------------------------------------|---------------------------------------------------------|
| Data atau input belum lengkap | Tolak, minta perbaikan, atau gunakan fallback | Menjaga kualitas tetapi dapat menambah friksi           |
| Hasil belum pasti             | Tampilkan ketidakpastian dan minta review     | Lebih aman tetapi membutuhkan waktu manusia             |
| Beban meningkat               | Scale, antrekan, atau pembatasan              | Menjaga layanan tetapi menambah biaya atau waktu tunggu |
| Perubahan berisiko            | Uji terbatas dan siapkan rollback             | Rilis lebih lambat tetapi dampak kegagalan lebih kecil  |

### Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan iam, secret, dan supply chain security.
- Menganggap identity sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

### Checkpoint

1. Jelaskan identity dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara role dan secret dalam bab ini?
3. Sebutkan satu risiko ketika iam, secret, dan supply chain security diterapkan tanpa pengujian.

### Latihan

1. Buat diagram sederhana yang menghubungkan identity, role, secret, signing. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.

## Bab 10 - Observability, Reliability, dan FinOps

### Tujuan Bab

Setelah mempelajari bab ini, peserta mampu:

- menjelaskan peran observability, reliability, dan finops dalam Infrastructure for AI;
- membedakan istilah yang sering tercampur;
- menyusun alur kerja sederhana yang dapat diuji;
- mengenali risiko, trade-off, dan kebutuhan dokumentasi.

### Gambaran Sederhana

Observability, Reliability, dan FinOps tidak berdiri sendiri. Bagian ini menghubungkan kebutuhan pengguna, proses kerja, data atau sumber daya, serta hasil yang akan dinilai. Pendekatan yang baik selalu dimulai dari pertanyaan: **keputusan apa yang ingin diperbaiki, bukti apa yang tersedia, dan siapa yang menanggung dampak ketika sistem salah?**

**Analogi:** Infrastruktur AI seperti fasilitas sebuah kota: model adalah aktivitas di dalamnya, sedangkan listrik, jalan, gudang, keamanan, dan pusat kendali menentukan apakah aktivitas dapat berlangsung stabil.

Pada praktiknya, kualitas bukan hanya berarti hasil tampak benar. Kualitas juga mencakup konsistensi, keterlacakan, keamanan, biaya, kemampuan dipelihara, dan kejelasan tindakan ketika terjadi kegagalan.

### Konsep Inti

| Istilah                | Penjelasan mudah                                                                                                   | Cara memeriksa pemahaman                                                                   |
|------------------------|--------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>SLI</b>             | konsep penting dalam observability, reliability, dan finops yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana SLI diukur, diuji, atau dikendalikan?             |
| <b>SLO</b>             | target terukur untuk tingkat layanan yang disepakati                                                               | Pertanyaan yang perlu dijawab: bagaimana SLO diukur, diuji, atau dikendalikan?             |
| <b>logging</b>         | konsep penting dalam observability, reliability, dan finops yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana logging diukur, diuji, atau dikendalikan?         |
| <b>tracing</b>         | konsep penting dalam observability, reliability, dan finops yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana tracing diukur, diuji, atau dikendalikan?         |
| <b>capacity</b>        | konsep penting dalam observability, reliability, dan finops yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana capacity diukur, diuji, atau dikendalikan?        |
| <b>cost allocation</b> | konsep penting dalam observability, reliability, dan finops yang perlu diberi definisi operasional sebelum dipakai | Pertanyaan yang perlu dijawab: bagaimana cost allocation diukur, diuji, atau dikendalikan? |

### Hubungan antarkonsep

Bayangkan alur **SLI** -> **SLO** -> **logging** -> **tracing**. Setiap panah menyatakan asumsi. Asumsi tersebut harus ditulis dan diuji. Ketika hasil akhir salah, tim dapat menelusuri pada tahap mana asumsi tidak lagi berlaku.

Contohnya, perubahan pada **SLI** dapat memengaruhi **SLO**, lalu mengubah cara **logging** dihitung. Tanpa pengamatan dan versioning, perubahan ini mudah dianggap sebagai perilaku pengguna atau performa model, padahal sumbernya berada di proses sebelumnya.



## Langkah Kerja

1. Tentukan tujuan observability, reliability, dan finops dan keputusan yang akan dipengaruhi.
2. Petakan input, output, pemilik, pengguna, serta batas sistem.
3. Tetapkan definisi dan kriteria penerimaan untuk SLI serta SLO.
4. Bangun versi kecil menggunakan data atau skenario yang aman.
5. Uji hasil, kegagalan, kelompok khusus, dan kondisi ekstrem.
6. Dokumentasikan keputusan, bukti, keterbatasan, serta tindak lanjut.

## Contoh Kasus

Dalam mini project **Platform Layanan AI Internal**, tim perlu menerapkan observability, reliability, dan finops secara bertahap. Mulailah dengan satu alur yang kecil dan dapat diamati. Catat input, transformasi atau keputusan, output, waktu proses, serta alasan ketika suatu item ditolak.

Misalnya, tim menemukan bahwa **SLI** tidak konsisten. Respons yang buruk adalah langsung menambah kompleksitas. Respons yang lebih baik adalah mengisolasi sumber masalah, membuat aturan validasi, mengukur frekuensi kejadian, dan menentukan siapa yang bertanggung jawab memperbaikinya.

| Situasi                       | Pilihan tindakan                              | Trade-off                                               |
|-------------------------------|-----------------------------------------------|---------------------------------------------------------|
| Data atau input belum lengkap | Tolak, minta perbaikan, atau gunakan fallback | Menjaga kualitas tetapi dapat menambah friksi           |
| Hasil belum pasti             | Tampilkan ketidakpastian dan minta review     | Lebih aman tetapi membutuhkan waktu manusia             |
| Beban meningkat               | Scale, antrekan, atau pembatasan              | Menjaga layanan tetapi menambah biaya atau waktu tunggu |
| Perubahan berisiko            | Uji terbatas dan siapkan rollback             | Rilis lebih lambat tetapi dampak kegagalan lebih kecil  |

## Kesalahan Umum

- Memulai dari alat sebelum memahami tujuan observability, reliability, dan finops.
- Menganggap SLI sudah memiliki definisi yang sama bagi semua pihak.
- Hanya menguji jalur normal dan mengabaikan kegagalan atau data yang tidak lengkap.
- Tidak menetapkan pemilik, indikator, dan prosedur ketika kualitas menurun.

## Checkpoint

1. Jelaskan SLI dengan kalimat sendiri dan berikan satu contoh.
2. Apa hubungan antara SLO dan logging dalam bab ini?
3. Sebutkan satu risiko ketika observability, reliability, dan finops diterapkan tanpa pengujian.

## Latihan

1. Buat diagram sederhana yang menghubungkan SLI, SLO, logging, tracing. Tandai asumsi dan titik kegagalan.
2. Tulis kriteria penerimaan untuk satu proses dalam bab ini. Sertakan kondisi normal, error, dan kasus batas.
3. Pilih satu risiko dan buat kontrol preventif, kontrol detektif, serta prosedur respons.



## Bab 11 - Mini Project: Platform Layanan AI Internal

### Tujuan Proyek

Merancang platform untuk training dan inference dengan container, penyimpanan terpisah, IAM, observability, backup, dan estimasi biaya.

Proyek harus cukup kecil untuk selesai, tetapi cukup lengkap untuk memperlihatkan alur berpikir. Nilai utama bukan pada tampilan atau kompleksitas, melainkan kejelasan tujuan, kualitas keputusan, pengujian, dan dokumentasi.

### Deliverable

- problem statement dan intended use;
- diagram arsitektur atau alur kerja;
- data dictionary atau inventaris sumber;
- baseline yang dapat dijalankan;
- hasil pengujian normal, error, dan kasus batas;
- daftar risiko serta kontrol;
- ringkasan hasil untuk audiens nonteknis;
- README dan rencana pengembangan.

### Tahapan Pengerjaan

1. Tulis masalah, pengguna, keputusan, dan batas penggunaan.
2. Buat inventaris data, sumber daya, pemilik, izin, serta risiko.
3. Rancang arsitektur atau alur kerja versi minimum.
4. Bangun baseline yang dapat dijalankan ulang.
5. Tambahkan validasi, logging, dan penanganan error.
6. Susun golden set atau skenario uji, termasuk kasus batas.
7. Ukur outcome, guardrail, performa, dan biaya.
8. Dokumentasikan hasil, keterbatasan, runbook, serta rencana iterasi.

### Contoh Kode Awal

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app ./app
RUN useradd --create-home appuser
USER appuser
EXPOSE 8000
CMD ["python", "-m", "app"]
```

Kode tersebut hanya titik awal. Tambahkan pemeriksaan input, konfigurasi, test, logging, serta penanganan error sesuai konteks proyek.

### Rubrik Penilaian

| Aspek           | Bobot | Indikator                                           |
|-----------------|-------|-----------------------------------------------------|
| Problem framing | 15%   | Pengguna, keputusan, outcome, dan batas jelas       |
| Data atau input | 15%   | Sumber, izin, kualitas, dan representasi dijelaskan |
| Rancangan       | 15%   | Komponen, interface, serta trade-off masuk akal     |
| Implementasi    | 20%   | Baseline berjalan, modular, dan dapat diulang       |
| Evaluasi        | 15%   | Metrik, kasus batas, serta subgroup diperiksa       |

| Aspek                | Bobot | Indikator                                     |
|----------------------|-------|-----------------------------------------------|
| Responsible practice | 10%   | Privasi, keamanan, fairness, dan human review |
| Komunikasi           | 10%   | Dokumentasi dan demo mudah dipahami           |

### Pertanyaan Review Proyek

1. Keputusan apa yang benar-benar dibantu proyek ini?
2. Apa kegagalan yang paling mungkin dan paling berbahaya?
3. Bukti apa yang menunjukkan solusi lebih baik daripada baseline?
4. Siapa yang berwenang menyetujui, menghentikan, atau mengubah sistem?
5. Informasi apa yang harus terlihat oleh pengguna agar tidak salah memahami hasil?

## Bab 12 - Kuis Akhir

Pilih jawaban yang paling tepat.

1. Ketika memulai **Arsitektur Infrastruktur AI**, tindakan yang paling tepat adalah ...
  - A. Mendefinisikan tujuan, konteks, dan cara menguji workload
  - B. Memilih alat paling baru tanpa memeriksa kebutuhan
  - C. Menghapus seluruh review agar proses lebih cepat
  - D. Mengukur keberhasilan hanya dari jumlah fitur
2. Ketika memulai **CPU, GPU, Accelerator, dan Memori**, tindakan yang paling tepat adalah ...
  - A. Memilih alat paling baru tanpa memeriksa kebutuhan
  - B. Mendefinisikan tujuan, konteks, dan cara menguji CPU
  - C. Menghapus seluruh review agar proses lebih cepat
  - D. Mengukur keberhasilan hanya dari jumlah fitur
3. Ketika memulai **Storage dan Siklus Hidup Data**, tindakan yang paling tepat adalah ...
  - A. Memilih alat paling baru tanpa memeriksa kebutuhan
  - B. Menghapus seluruh review agar proses lebih cepat
  - C. Mendefinisikan tujuan, konteks, dan cara menguji object storage
  - D. Mengukur keberhasilan hanya dari jumlah fitur
4. Ketika memulai **Jaringan untuk Workload AI**, tindakan yang paling tepat adalah ...
  - A. Memilih alat paling baru tanpa memeriksa kebutuhan
  - B. Menghapus seluruh review agar proses lebih cepat
  - C. Mengukur keberhasilan hanya dari jumlah fitur
  - D. Mendefinisikan tujuan, konteks, dan cara menguji latency
5. Ketika memulai **Virtualization dan Container**, tindakan yang paling tepat adalah ...
  - A. Mendefinisikan tujuan, konteks, dan cara menguji virtual machine
  - B. Memilih alat paling baru tanpa memeriksa kebutuhan
  - C. Menghapus seluruh review agar proses lebih cepat
  - D. Mengukur keberhasilan hanya dari jumlah fitur
6. Ketika memulai **Cloud, On-Premise, dan Hybrid**, tindakan yang paling tepat adalah ...
  - A. Memilih alat paling baru tanpa memeriksa kebutuhan
  - B. Mendefinisikan tujuan, konteks, dan cara menguji cloud
  - C. Menghapus seluruh review agar proses lebih cepat
  - D. Mengukur keberhasilan hanya dari jumlah fitur
7. Ketika memulai **Orkestrasi dengan Kubernetes**, tindakan yang paling tepat adalah ...
  - A. Memilih alat paling baru tanpa memeriksa kebutuhan
  - B. Menghapus seluruh review agar proses lebih cepat
  - C. Mendefinisikan tujuan, konteks, dan cara menguji pod
  - D. Mengukur keberhasilan hanya dari jumlah fitur
8. Ketika memulai **Infrastructure as Code**, tindakan yang paling tepat adalah ...
  - A. Memilih alat paling baru tanpa memeriksa kebutuhan
  - B. Menghapus seluruh review agar proses lebih cepat
  - C. Mengukur keberhasilan hanya dari jumlah fitur
  - D. Mendefinisikan tujuan, konteks, dan cara menguji declarative
9. Ketika memulai **IAM, Secret, dan Supply Chain Security**, tindakan yang paling tepat adalah ...
  - A. Mendefinisikan tujuan, konteks, dan cara menguji identity
  - B. Memilih alat paling baru tanpa memeriksa kebutuhan
  - C. Menghapus seluruh review agar proses lebih cepat
  - D. Mengukur keberhasilan hanya dari jumlah fitur
10. Ketika memulai **Observability, Reliability, dan FinOps**, tindakan yang paling tepat adalah ...
  - A. Memilih alat paling baru tanpa memeriksa kebutuhan
  - B. Mendefinisikan tujuan, konteks, dan cara menguji SLI
  - C. Menghapus seluruh review agar proses lebih cepat
  - D. Mengukur keberhasilan hanya dari jumlah fitur

11. Input penting berubah tanpa pemberitahuan. Kontrol terbaik adalah ...
- A. menambah kompleksitas tanpa diagnosis
  - B. menyembunyikan error dari pengguna
  - C. Kontrak, validasi, versioning, dan alert
  - D. menghapus logging untuk menghemat ruang
12. Hasil sistem terlihat baik pada data latihan tetapi buruk pada konteks baru. Langkah terbaik adalah ...
- A. menambah kompleksitas tanpa diagnosis
  - B. menyembunyikan error dari pengguna
  - C. menghapus logging untuk menghemat ruang
  - D. Memeriksa split, leakage, representasi data, dan generalisasi
13. Tim tidak tahu siapa yang harus merespons alarm. Perbaikan utama adalah ...
- A. Menetapkan ownership, severity, runbook, dan escalation
  - B. menambah kompleksitas tanpa diagnosis
  - C. menyembunyikan error dari pengguna
  - D. menghapus logging untuk menghemat ruang
14. Perubahan besar akan diluncurkan. Cara menurunkan blast radius adalah ...
- A. menambah kompleksitas tanpa diagnosis
  - B. Rilis bertahap, observasi, dan rollback
  - C. menyembunyikan error dari pengguna
  - D. menghapus logging untuk menghemat ruang
15. Pengguna tidak memahami keterbatasan hasil. Desain yang tepat adalah ...
- A. menambah kompleksitas tanpa diagnosis
  - B. menyembunyikan error dari pengguna
  - C. Menjelaskan sumber, ketidakpastian, batas penggunaan, dan koreksi
  - D. menghapus logging untuk menghemat ruang
16. Data sensitif digunakan untuk latihan. Prinsip pertama adalah ...
- A. menambah kompleksitas tanpa diagnosis
  - B. menyembunyikan error dari pengguna
  - C. menghapus logging untuk menghemat ruang
  - D. Minimization, izin, kontrol akses, dan penggunaan data sintetis bila memungkinkan
17. Satu metrik meningkat tetapi dampak keseluruhan memburuk. Tim perlu ...
- A. Menggunakan outcome metric dan guardrail secara bersamaan
  - B. menambah kompleksitas tanpa diagnosis
  - C. menyembunyikan error dari pengguna
  - D. menghapus logging untuk menghemat ruang
18. Sistem gagal pada sebagian kelompok. Respons yang benar adalah ...
- A. menambah kompleksitas tanpa diagnosis
  - B. Menganalisis subgroup, dampak, akar masalah, dan mitigasi
  - C. menyembunyikan error dari pengguna
  - D. menghapus logging untuk menghemat ruang
19. Biaya naik tanpa kenaikan nilai. Langkah awal adalah ...
- A. menambah kompleksitas tanpa diagnosis
  - B. menyembunyikan error dari pengguna
  - C. Mengukur penggunaan per komponen dan menghubungkannya dengan outcome
  - D. menghapus logging untuk menghemat ruang
20. Temuan belum cukup kuat untuk keputusan otomatis. Pilihan aman adalah ...
- A. menambah kompleksitas tanpa diagnosis
  - B. menyembunyikan error dari pengguna
  - C. menghapus logging untuk menghemat ruang
  - D. Membatasi penggunaan sebagai dukungan keputusan dan menambahkan human review

### Kunci dan Pembahasan

| No. | Jawaban | Pembahasan                         |
|-----|---------|------------------------------------|
| 1   | A       | Pendekatan dimulai dari tujuan dan |

| No. | Jawaban | Pembahasan                                                                                                       |
|-----|---------|------------------------------------------------------------------------------------------------------------------|
|     |         | definisi operasional workload, kemudian alat dipilih sesuai kebutuhan.                                           |
| 2   | B       | Pendekatan dimulai dari tujuan dan definisi operasional CPU, kemudian alat dipilih sesuai kebutuhan.             |
| 3   | C       | Pendekatan dimulai dari tujuan dan definisi operasional object storage, kemudian alat dipilih sesuai kebutuhan.  |
| 4   | D       | Pendekatan dimulai dari tujuan dan definisi operasional latency, kemudian alat dipilih sesuai kebutuhan.         |
| 5   | A       | Pendekatan dimulai dari tujuan dan definisi operasional virtual machine, kemudian alat dipilih sesuai kebutuhan. |
| 6   | B       | Pendekatan dimulai dari tujuan dan definisi operasional cloud, kemudian alat dipilih sesuai kebutuhan.           |
| 7   | C       | Pendekatan dimulai dari tujuan dan definisi operasional pod, kemudian alat dipilih sesuai kebutuhan.             |
| 8   | D       | Pendekatan dimulai dari tujuan dan definisi operasional declarative, kemudian alat dipilih sesuai kebutuhan.     |
| 9   | A       | Pendekatan dimulai dari tujuan dan definisi operasional identity, kemudian alat dipilih sesuai kebutuhan.        |
| 10  | B       | Pendekatan dimulai dari tujuan dan definisi operasional SLI, kemudian alat dipilih sesuai kebutuhan.             |
| 11  | C       | Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.                       |
| 12  | D       | Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.                       |
| 13  | A       | Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.                       |
| 14  | B       | Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.                       |
| 15  | C       | Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.                       |
| 16  | D       | Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti.                       |

| No. | Jawaban  | Pembahasan                                                                                 |
|-----|----------|--------------------------------------------------------------------------------------------|
| 17  | <b>A</b> | Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti. |
| 18  | <b>B</b> | Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti. |
| 19  | <b>C</b> | Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti. |
| 20  | <b>D</b> | Pilihan tersebut membuat masalah dapat dilacak, dibatasi, dan ditangani berdasarkan bukti. |

#### Interpretasi Skor

- **17-20 benar:** siap mengerjakan proyek dengan pengawasan ringan.
- **13-16 benar:** pemahaman baik, ulangi bagian yang masih lemah.
- **9-12 benar:** pelajari kembali konsep inti dan latihan.
- **0-8 benar:** ulangi modul secara bertahap dengan mentor atau teman belajar.

## Bab 13 - Diskusi dan Refleksi

---

### Pertanyaan Diskusi

1. Bagian mana dari Infrastructure for AI yang paling mudah diremehkan tetapi berpotensi menimbulkan kegagalan besar?
2. Kapan solusi sederhana lebih baik daripada solusi yang lebih otomatis atau kompleks?
3. Metrik apa yang dapat mendorong perilaku salah bila digunakan sendirian?
4. Bagaimana pengguna dapat mengoreksi sistem dan melihat tindak lanjutnya?
5. Siapa yang menerima manfaat dan siapa yang mungkin menerima risiko?

### Jurnal Refleksi

Tuliskan satu halaman yang menjawab:

- konsep yang paling mengubah cara berpikir;
- asumsi yang sebelumnya dianggap benar;
- kesalahan yang terjadi ketika latihan;
- bukti yang masih kurang;
- satu tindakan belajar berikutnya.

## Bab 14 - Glosarium dan Checklist

### Glosarium

| Istilah                           | Makna ringkas                                                                                      |
|-----------------------------------|----------------------------------------------------------------------------------------------------|
| <b>workload</b>                   | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>training</b>                   | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>inference</b>                  | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>control plane</b>              | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>data plane</b>                 | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>non-functional requirement</b> | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>CPU</b>                        | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>GPU</b>                        | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>accelerator</b>                | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>VRAM</b>                       | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>bandwidth</b>                  | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>precision</b>                  | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>object storage</b>             | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>block</b>                      | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>file</b>                       | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>IOPS</b>                       | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>throughput</b>                 | jumlah pekerjaan yang dapat diselesaikan per satuan waktu.                                         |
| <b>tiering</b>                    | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>latency</b>                    | waktu yang dibutuhkan sistem untuk menghasilkan respons.                                           |
| <b>topology</b>                   | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>load balancer</b>              | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>egress</b>                     | konsep penting dalam infrastructure for ai yang perlu                                              |



| Istilah                      | Makna ringkas                                                                                      |
|------------------------------|----------------------------------------------------------------------------------------------------|
|                              | diberi definisi operasional sebelum dipakai.                                                       |
| <b>private network</b>       | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>virtual machine</b>       | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>container</b>             | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>image</b>                 | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>runtime</b>               | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>isolation</b>             | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>registry</b>              | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>cloud</b>                 | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>on-premise</b>            | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>hybrid</b>                | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>elasticity</b>            | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>sovereignty</b>           | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>shared responsibility</b> | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>pod</b>                   | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>deployment</b>            | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>service</b>               | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>job</b>                   | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>autoscaling</b>           | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>resource quota</b>        | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>declarative</b>           | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>Terraform</b>             | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>module</b>                | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |
| <b>state</b>                 | konsep penting dalam infrastructure for ai yang perlu diberi definisi operasional sebelum dipakai. |

## Checklist Sebelum Implementasi

- [ ] Masalah, pengguna, dan keputusan telah dinyatakan.
- [ ] Intended use dan penggunaan yang dilarang telah ditulis.
- [ ] Sumber data atau input memiliki izin dan provenance.
- [ ] Baseline dan kriteria penerimaan tersedia.
- [ ] Kondisi error, data kosong, dan kasus ekstrem diuji.
- [ ] Privasi, keamanan, bias, dan aksesibilitas ditinjau.
- [ ] Log, metric, owner, alert, dan runbook ditetapkan.
- [ ] Perubahan memiliki versioning, approval, dan rollback.
- [ ] Hasil dikomunikasikan bersama keterbatasannya.
- [ ] Jadwal review dan penghentian sistem ditentukan.

## Bab 15 - Ringkasan dan Referensi

---

### Ringkasan Cepat

1. Mulai dari keputusan dan pengguna, bukan alat.
2. Definisikan istilah, data, outcome, dan guardrail secara operasional.
3. Bangun baseline kecil yang dapat diamati dan diuji ulang.
4. Perlakukan keamanan, privasi, aksesibilitas, serta human review sebagai bagian desain.
5. Uji kondisi normal, kegagalan, subgroup, perubahan konteks, dan biaya.
6. Dokumentasikan provenance, asumsi, versi, owner, serta keterbatasan.
7. Gunakan monitoring dan feedback untuk memutuskan iterasi, rollback, atau penghentian.

### Referensi Utama

- Site Reliability Engineering - Google.
- Kubernetes Documentation.
- Terraform Documentation.
- NIST Cybersecurity Framework.
- FinOps Foundation Framework.

### Penutup

Kemampuan dalam Infrastructure for AI tidak hanya ditunjukkan oleh banyaknya alat yang dikuasai. Kemampuan terlihat dari cara peserta merumuskan masalah, memilih pendekatan yang proporsional, menguji klaim, melindungi pengguna, dan menjaga sistem tetap dapat dipertanggungjawabkan.

**HerAI Fellowship - Participant Module Versi 1.0.0 - 23 Juli 2026**